

Oracle Database — Sistema de Control de Inventario (Empresa Crispy)

Documento de apoyo académico

Ruiz Santiago Fernanda · Valdivia Hernández Angel Santiago · Zavala Morales Amy



Portada del proyecto: Sistema de control de inventario – Empresa Crispy, sobre Oracle Database.

1. Introducción

El presente documento desarrolla, con fines de apoyo académico, el diseño y la implementación de un sistema de control de inventario para una empresa ficticia denominada “Crispy”, dedicada a la comercialización de productos derivados del amaranto. El sistema utiliza Oracle Database como motor de base de datos para el almacenamiento estructurado de usuarios, productos, movimientos de stock y registros de compra-venta, y se apoya en una arquitectura cliente-servidor con un backend que expone una API REST consumida por un frontend web.

A lo largo del documento se exponen los siguientes aspectos: una introducción histórica a Oracle como compañía y como motor de base de datos; el proceso de instalación de Oracle Database Express Edition; los conceptos fundamentales de qué es, cómo funciona y para qué sirve un sistema gestor de bases de datos relacional como Oracle; los diagramas de arquitectura general, despliegue, secuencia e iteración del sistema; el diagrama de entidad-relación que modela la base de datos; un diccionario de datos;

fragmentos de código que ilustran la lógica del servidor; y finalmente, capturas de pantalla de la aplicación funcionando en sus distintos roles de usuario.

2. Historia de Oracle

Oracle Corporation es una de las compañías de software empresarial más influyentes en la historia de la computación, reconocida principalmente por haber popularizado el modelo de bases de datos relacionales en el mundo comercial. La compañía fue fundada en 1977 por Larry Ellison junto con Bob Miner y Ed Oates, originalmente bajo el nombre de Software Development Laboratories, antes de adoptar más adelante el nombre de su producto insignia, Oracle.

Uno de los primeros contratos de la naciente compañía fue el desarrollo de un sistema gestor de bases de datos para la CIA, en un equipo de menos de diez empleados. La apuesta por construir ese sistema sobre el modelo relacional —teorizado por Edgar F. Codd en IBM— y consultable mediante el lenguaje SQL resultó decisiva: en 1981 IBM se convirtió en cliente de Oracle, lo que impulsó que las ventas de la startup se duplicaran cada año durante los siguientes siete años.

La compañía salió a bolsa en 1986 y, un año después, se convirtió en la mayor empresa de gestión de bases de datos del mundo. Atravesó después una etapa de dificultades financieras a raíz de irregularidades contables detectadas en una auditoría interna, lo que provocó una caída abrupta en el valor de sus acciones; sin embargo, tras una reestructuración de la dirección, Oracle recuperó su solidez financiera hacia finales de 1992, consolidándose nuevamente como referente del sector.

Con el paso de las décadas, Oracle Database evolucionó incorporando funcionalidades cada vez más sofisticadas: procesamiento de transacciones distribuidas, particionado de tablas, replicación, procesamiento en paralelo, soporte para datos semiestructurados (XML, JSON), capacidades analíticas avanzadas y, más recientemente, integración con servicios de nube e inteligencia artificial. Hoy en día, Oracle Database es ampliamente utilizado en sectores que requieren alta disponibilidad, integridad transaccional estricta y manejo de grandes volúmenes de datos, como el sector bancario, financiero, gubernamental y de telecomunicaciones.



Capacidades de manejo de datos de Oracle Database: importación tabular, JSON, exportación estructurada, datos geoespaciales, relaciones tipo grafo y almacenamiento persistente.

2.1 Hitos relevantes en la evolución de Oracle

- **Fundación en 1977:** Larry Ellison, Bob Miner y Ed Oates fundan Software Development Laboratories, compañía que más adelante adoptaría el nombre de su producto principal, Oracle.
- **Adopción temprana por IBM (1981):** este acuerdo validó comercialmente el modelo relacional consultable mediante SQL y permitió que las ventas de la compañía se duplicaran año tras año durante varios años.

- **Salida a bolsa y liderazgo de mercado (1986-1987):** Oracle realizó su oferta pública inicial en 1986 y, un año después, se convirtió en la mayor empresa de gestión de bases de datos del mundo.
- **Recuperación financiera (1992):** tras una etapa de dificultades contables, la compañía se reestructuró y recuperó su solidez financiera, retomando el liderazgo del mercado de bases de datos.
- **Escalabilidad empresarial:** funcionalidades como el particionado de tablas, el procesamiento en paralelo (Oracle RAC) y la replicación entre centros de datos permiten que Oracle Database opere a gran escala en organizaciones globales.
- **Ediciones para distintos contextos:** Oracle ofrece ediciones diferenciadas, desde Oracle Database Express Edition (XE), gratuita y orientada a aprendizaje y pequeños proyectos, hasta Oracle Database Enterprise Edition, pensada para cargas de producción críticas.
- **Evolución hacia la nube:** las versiones más recientes de Oracle integran capacidades de nube (Oracle Cloud Infrastructure, Oracle Autonomous Database), automatizando tareas de administración como ajuste de rendimiento, parches y copias de seguridad.

Relevancia para este proyecto

Para el sistema de control de inventario de la Empresa Crispy, se utiliza Oracle Database Express Edition (XE), una edición gratuita pensada para fines educativos y de desarrollo, que conserva el motor relacional completo de Oracle con límites de tamaño de base de datos y de recursos de hardware.

3. Instalación de Oracle Database

Para el desarrollo del proyecto se utiliza Oracle Database Express Edition (XE), una edición gratuita que conserva el motor relacional completo de Oracle, con límites razonables de tamaño de base de datos y de uso de memoria y CPU, pensada específicamente para fines de aprendizaje, desarrollo y proyectos de baja escala. El proceso de instalación, aunque varía ligeramente según el sistema operativo, sigue un flujo general de cinco etapas que se describe a continuación.



Flujo general de instalación de Oracle Database: descarga del software, ejecución del instalador, configuración de parámetros, definición de contraseña y finalización/verificación.

3.1 Paso 1 — Descarga del software

El proceso inicia accediendo al sitio web oficial de Oracle para descargar el instalador correspondiente. Es necesario seleccionar la versión de Oracle Database XE adecuada para el sistema operativo utilizado (Windows o Linux), aceptar los términos de la licencia de uso y proceder con la descarga del archivo de instalación, que generalmente se distribuye en formato comprimido.

3.2 Paso 2 — Ejecución del instalador

Una vez descargado el archivo, se debe descomprimir para acceder a sus contenidos, entre los cuales se encuentra el ejecutable setup.exe (en el caso de Windows). Es importante ejecutar este instalador con privilegios de administrador, ya que el proceso de instalación requiere registrar servicios del sistema operativo y reservar puertos de red.

3.3 Paso 3 — Configuración de parámetros

Durante esta etapa, el asistente de instalación solicita aceptar el acuerdo de licencia del software y seleccionar la ruta de instalación. Por defecto, Oracle Database XE se instala en una ruta como C:\app\, aunque esta puede personalizarse según las preferencias del usuario o las políticas de la organización.

3.4 Paso 4 — Definición de contraseña

Uno de los pasos más sensibles del proceso es la creación y confirmación de la contraseña para las cuentas administrativas del sistema, principalmente SYS y SYSTEM, que cuentan con privilegios elevados sobre toda la instancia de base de datos. Es fundamental anotar esta contraseña en un lugar seguro, ya que su pérdida puede complicar significativamente el acceso administrativo posterior.

3.5 Paso 5 — Finalización y verificación

En la etapa final, el instalador completa la configuración del entorno, inicia los servicios necesarios y permite verificar que la instalación fue exitosa. Resulta importante revisar que los puertos de red configurados —por ejemplo, el puerto 1521 para el listener de la base de datos— estén correctamente asignados y disponibles, y confirmar que los servicios de Oracle se encuentren iniciados antes de intentar establecer una conexión desde una aplicación cliente.

3.6 Verificación posterior a la instalación

Tras finalizar la instalación, es recomendable validar el correcto funcionamiento del motor mediante una conexión de prueba, ya sea utilizando una herramienta gráfica como Oracle SQL Developer o mediante la utilidad de línea de comandos SQL*Plus.

```
-- Conexión de prueba mediante SQL*Plus
sqlplus system/contraseña@localhost:1521/XEPDB1

-- Si la conexión es exitosa, se puede verificar la versión del motor:
SELECT * FROM v$version;
```

Paso	Acción principal	Consideración clave
1	Descarga del software	Verificar compatibilidad con el sistema operativo (Windows/Linux)
2	Ejecución del instalador	Ejecutar con privilegios de administrador
3	Configuración de parámetros	Aceptar licencia y definir ruta de instalación
4	Definición de contraseña	Anotar y resguardar las credenciales de SYS/SYSTEM
5	Finalización y verificación	Confirmar puertos de red (1521) y servicios activos

4. ¿Qué es, cómo funciona y para qué es Oracle Database?

Antes de profundizar en la arquitectura específica del sistema desarrollado para la Empresa Crispy, conviene responder a tres preguntas conceptuales que enmarcan el resto del documento: qué es Oracle Database, cómo funciona internamente y para qué sirve dentro de un sistema de información como el que se está construyendo.



Tres preguntas conceptuales sobre Oracle Database aplicadas al proyecto Crispy: qué es, cómo funciona y para qué sirve.

4.1 ¿Qué es Oracle Database?

Oracle Database es un sistema de gestión de bases de datos relacionales (RDBMS, por sus siglas en inglés), es decir, un software encargado de almacenar, organizar y administrar datos estructurados en forma de tablas compuestas por filas y columnas, manteniendo relaciones lógicas entre ellas mediante claves primarias y foráneas. A diferencia de simplemente guardar información en archivos sueltos, un RDBMS como Oracle garantiza propiedades fundamentales para la integridad de los datos, conocidas como propiedades ACID: Atomicidad, Consistencia, Aislamiento y Durabilidad.

4.2 ¿Cómo funciona Oracle Database?

Internamente, Oracle Database opera mediante una instancia (el conjunto de procesos en memoria y procesos de fondo que gestionan la actividad de la base de datos) y uno o varios archivos de base de datos en disco, donde se almacenan físicamente los datos, los índices y los metadatos del esquema. Cuando una aplicación cliente envía una instrucción SQL —por ejemplo, un INSERT para registrar una nueva venta—, esta solicitud es recibida por el listener de red de Oracle, procesada por la instancia, y finalmente persistida en los archivos de datos, todo ello mientras se registran los cambios en archivos de redo log que permiten recuperar la base de datos ante una falla.

Las consultas y modificaciones de datos se expresan en SQL, el lenguaje estándar para interactuar con bases de datos relacionales, complementado en el caso de Oracle por PL/SQL, su lenguaje procedimental propietario, que permite escribir lógica de negocio (condicionales, ciclos, funciones, procedimientos almacenados) directamente dentro del motor de base de datos.

4.3 ¿Para qué es Oracle Database en este proyecto?

En el contexto del sistema de control de inventario de la Empresa Crispy, Oracle Database cumple el rol de fuente única de verdad (single source of truth) para toda la información crítica del negocio: los usuarios del sistema y sus roles, el catálogo de productos con su stock disponible, y el historial de movimientos de compra y venta. Cada vez que un comprador realiza una venta o un distribuidor registra una entrada de materia prima, esa operación se traduce en instrucciones SQL que actualizan de forma consistente las tablas correspondientes, garantizando que el inventario reflejado en el sistema sea siempre preciso y confiable.

Pregunta	Respuesta resumida
¿Qué es?	Un sistema gestor de bases de datos relacionales que almacena datos en tablas con integridad garantizada (ACID)
¿Cómo funciona?	Mediante una instancia en memoria, archivos de datos en disco y el lenguaje SQL/PL-SQL para consultar y modificar la información
¿Para qué es?	Para ser la fuente única de verdad del inventario, usuarios y movimientos del sistema Crispy

5. Diagrama de Arquitectura General

El diagrama de arquitectura general describe, a un nivel conceptual, cómo se organizan los distintos actores y componentes del sistema de la Empresa Crispy, desde el usuario hasta la base de datos. El flujo se estructura en capas claramente diferenciadas, lo que facilita tanto la comprensión del sistema como su mantenimiento futuro.

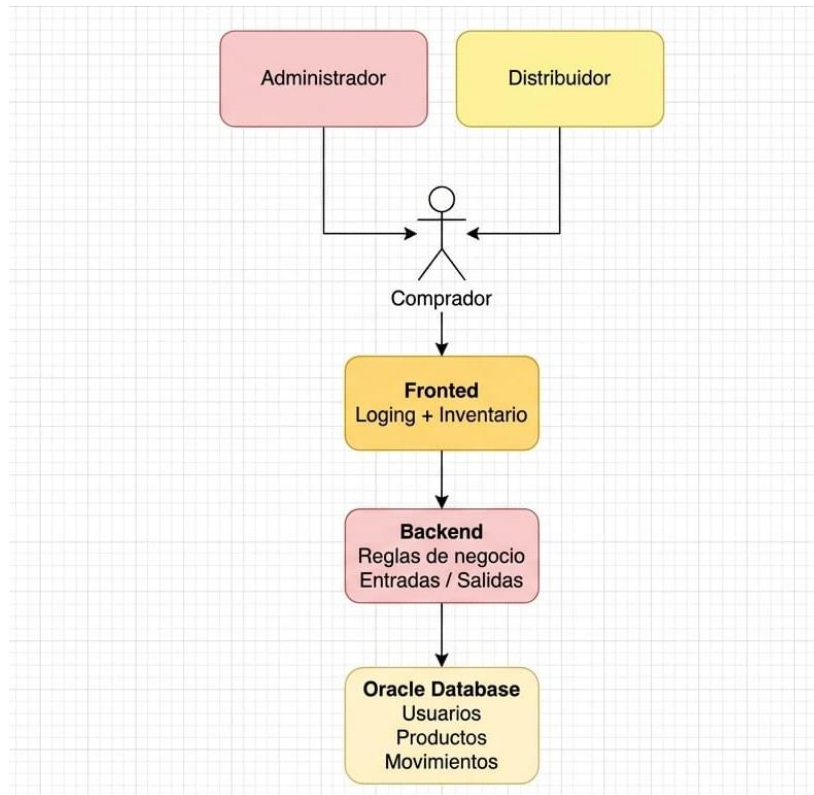


Diagrama de arquitectura general del sistema Crispy: actores (Administrador, Distribuidor, Comprador), capa de Frontend, capa de Backend y Oracle Database.

5.1 Actores del sistema

- **Administrador:** actor con visión global del negocio, responsable de supervisar métricas financieras y operativas del sistema en su conjunto.
- **Distribuidor:** actor encargado de registrar entradas de materia prima o producto, incrementando el stock disponible en el inventario.
- **Comprador:** actor que interactúa directamente con el catálogo de productos para realizar compras, lo cual reduce el stock disponible y genera ingresos para el negocio.

5.2 Capas del sistema

Como se observa en el diagrama, el flujo de información atraviesa tres capas principales antes de llegar a la base de datos: en primer lugar, el Frontend, responsable del login de usuarios y de la gestión visual del inventario; en segundo lugar, el Backend, que concentra las reglas de negocio del sistema —como validar que existe stock suficiente antes de procesar una venta— y gestiona las entradas y salidas de productos; y finalmente, Oracle Database, que almacena de forma persistente los datos de usuarios, productos y movimientos.

Capa	Responsabilidad principal
Frontend	Interfaz visual; login de usuarios; presentación del inventario
Backend	Reglas de negocio; validaciones; procesamiento de entradas y salidas
Oracle Database	Almacenamiento persistente de usuarios, productos y movimientos

Por qué separar estas capas

Separar el sistema en capas de Frontend, Backend y base de datos permite que cada una evolucione de forma relativamente independiente: la interfaz visual puede rediseñarse sin tocar la lógica de negocio, y las reglas de negocio pueden modificarse sin alterar directamente la estructura de almacenamiento en Oracle Database.

6. Diagrama de Despliegue

Mientras que el diagrama de arquitectura general describe los componentes lógicos del sistema, el diagrama de despliegue representa cómo esos componentes se distribuyen físicamente sobre dispositivos y servidores reales, especificando qué se ejecuta en cada nodo de la infraestructura y cómo se comunican entre sí.

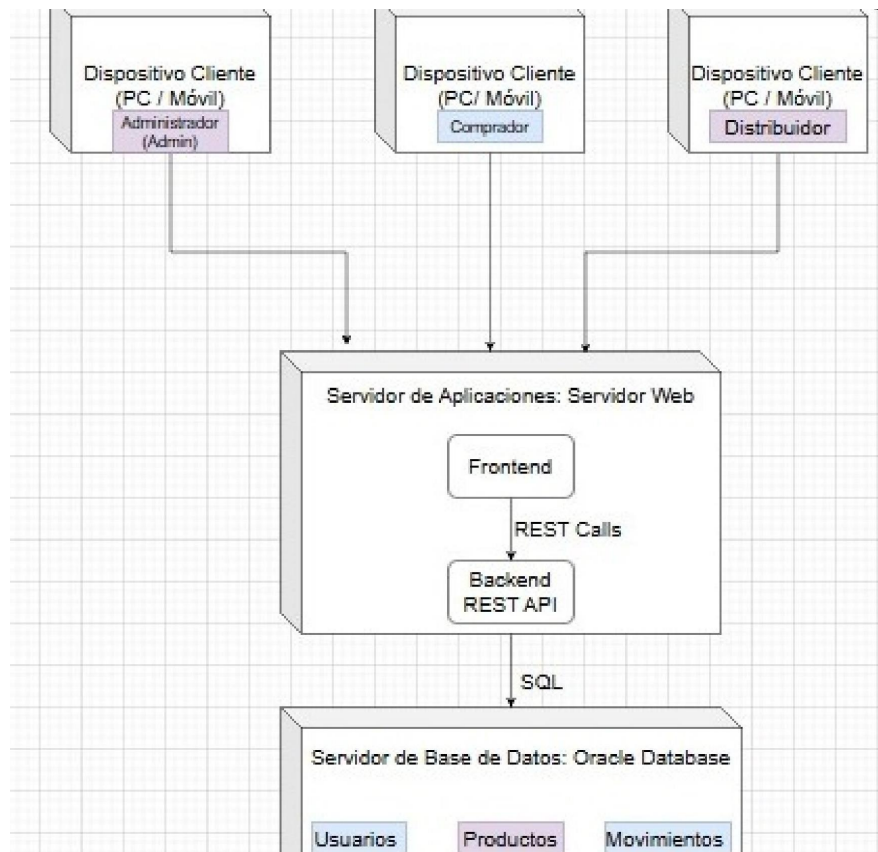


Diagrama de despliegue: dispositivos cliente de Administrador, Comprador y Distribuidor conectados a un Servidor de Aplicaciones (Frontend y Backend REST API), que a su vez se comunica vía SQL con el Servidor de Base de Datos Oracle Database.

6.1 Nodos de la infraestructura

- **Dispositivos cliente (PC / Móvil):** tres tipos de dispositivos cliente acceden al sistema, cada uno asociado a un rol específico: Administrador, Comprador o Distribuidor. Estos dispositivos ejecutan únicamente la interfaz visual que el usuario consume desde su navegador.
- **Servidor de Aplicaciones (Servidor Web):** este nodo concentra dos componentes: el Frontend, que sirve las páginas y la lógica de presentación, y el Backend (REST API), que recibe las peticiones desde el Frontend mediante llamadas REST (REST Calls) y aplica las reglas de negocio correspondientes.
- **Servidor de Base de Datos (Oracle Database):** este nodo alberga el motor de Oracle Database, donde se almacenan de forma persistente las tablas de Usuarios, Productos y Movimientos. El Backend se comunica con este servidor exclusivamente mediante instrucciones SQL.

6.2 Flujo de comunicación entre nodos

El flujo de comunicación sigue una secuencia estrictamente jerárquica: el dispositivo cliente nunca se comunica directamente con la base de datos, sino que toda solicitud pasa primero por el Frontend, que la traduce en una llamada REST hacia el Backend; el Backend, a su vez, traduce esa llamada en una o varias instrucciones SQL dirigidas a Oracle Database. Esta separación es una práctica recomendada de seguridad y de diseño, ya que evita que las credenciales de la base de datos o la lógica de acceso a datos queden expuestas directamente en el dispositivo del usuario final.

Origen	Destino	Protocolo / Mecanismo
Dispositivo Cliente	Servidor de Aplicaciones (Frontend)	HTTP / HTTPS
Frontend	Backend (REST API)	REST Calls
Backend (REST API)	Servidor de Base de Datos (Oracle Database)	SQL

7. Diagrama de Iteración

El diagrama de iteración (también conocido como diagrama de secuencia) detalla, paso a paso, el intercambio de mensajes entre el actor Comprador/Vendedor, el Frontend, la API, Oracle Database y el Frontend de confirmación, durante el proceso completo de registrar una venta en el sistema.

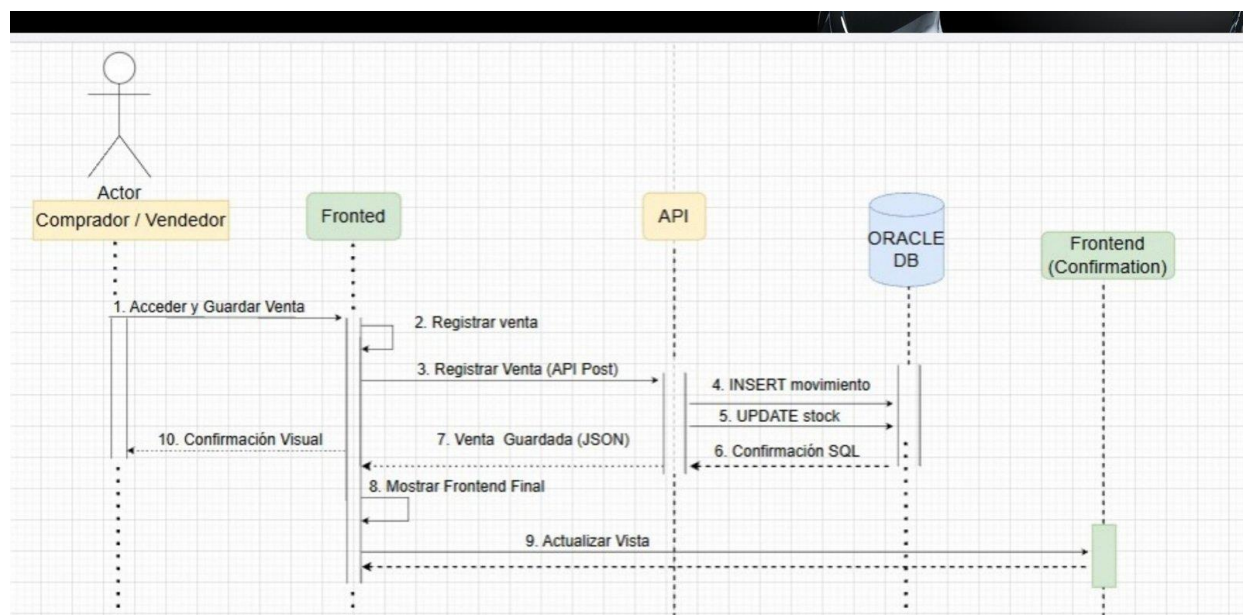


Diagrama de secuencia del proceso de venta: desde que el actor accede y guarda la venta hasta la confirmación visual final, pasando por el registro en la API y las operaciones INSERT/UPDATE en Oracle Database.

7.1 Descripción paso a paso del flujo de venta

- **1. Acceder y Guardar Venta:** el actor Comprador/Vendedor inicia el flujo solicitando al Frontend que registre una venta.
- **2. Registrar venta:** el Frontend ejecuta una operación interna para preparar los datos de la venta antes de enviarlos.
- **3. Registrar Venta (API Post):** el Frontend envía una petición HTTP POST hacia la API con los datos de la venta.
- **4. INSERT movimiento:** la API ejecuta una instrucción SQL INSERT contra Oracle Database para registrar el nuevo movimiento de venta.
- **5. UPDATE stock:** inmediatamente después, la API ejecuta una instrucción SQL UPDATE para descontar la cantidad vendida del stock disponible del producto.
- **6. Confirmación SQL:** Oracle Database confirma a la API que ambas operaciones (INSERT y UPDATE) se completaron correctamente.
- **7. Venta Guardada (JSON):** la API responde al Frontend con un objeto JSON confirmando que la venta fue guardada exitosamente.
- **8. Mostrar Frontend Final:** el Frontend procesa internamente la respuesta para preparar la actualización de la vista.
- **9. Actualizar Vista:** se notifica al componente de Frontend (Confirmation) para refrescar la información mostrada al usuario.
- **10. Confirmación Visual:** finalmente, el actor Comprador/Vendedor recibe una confirmación visual de que la venta fue procesada con éxito.

7.2 Importancia de este flujo

Este diagrama evidencia un principio importante de diseño transaccional: el registro del movimiento (INSERT) y la actualización del stock (UPDATE) ocurren como parte de la misma operación lógica, en el mismo paso de comunicación con la base de datos. En una implementación real con Oracle Database, ambas instrucciones deberían ejecutarse dentro de una misma transacción (utilizando COMMIT al finalizar ambas operaciones con éxito, o ROLLBACK si alguna de ellas falla), de modo que nunca quede registrado un movimiento de venta sin el correspondiente descuento de stock, ni viceversa.

8. Diagrama de Entidad-Relación

El diagrama de entidad-relación (DER) modela la estructura lógica de la base de datos del sistema Crispy, mostrando las entidades principales, sus atributos y las relaciones de cardinalidad que existen entre ellas. Este diagrama es la base directa sobre la cual se diseñan las tablas físicas en Oracle Database.

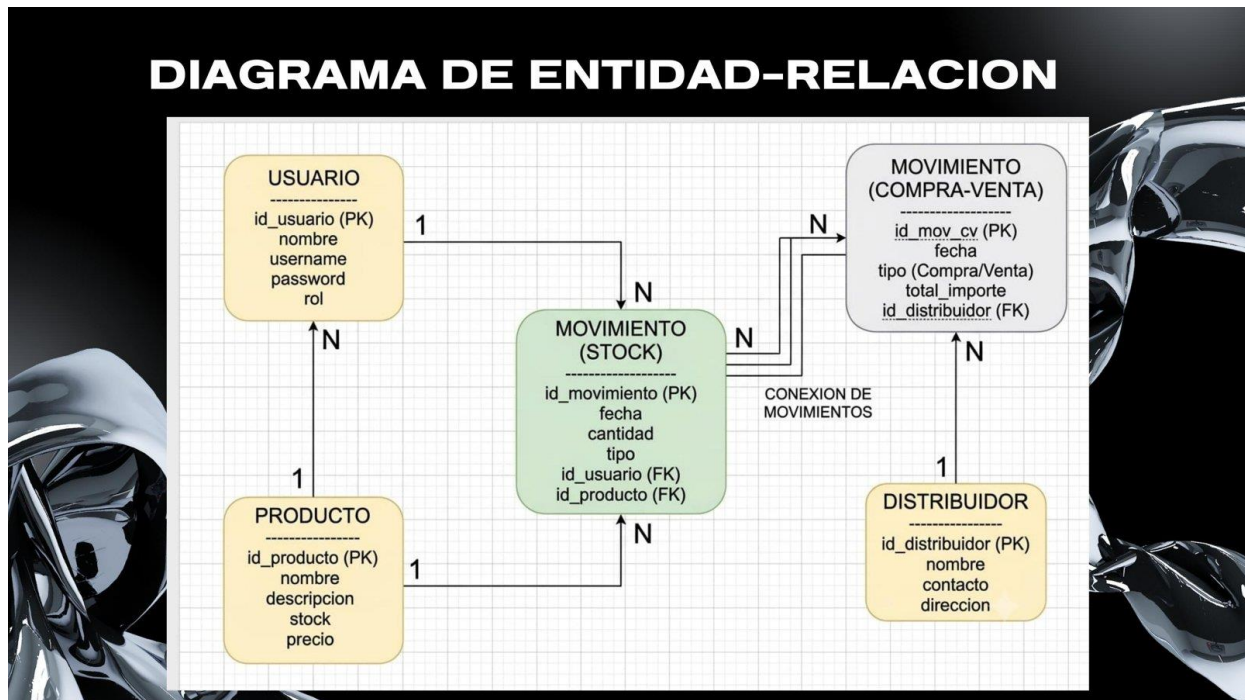


Diagrama de entidad-relación del sistema Crispy: USUARIO, PRODUCTO, MOVIMIENTO (STOCK), MOVIMIENTO (COMPRA-VENTA) y DISTRIBUIDOR, con sus respectivas claves primarias, foráneas y cardinalidades.

8.1 Entidades principales

- **USUARIO:** almacena los datos de acceso de cada persona que utiliza el sistema: `id_usuario` (PK), nombre, username, password y rol. El atributo rol determina si el usuario actúa como Administrador, Distribuidor o Comprador/Vendedor.
- **PRODUCTO:** representa el catálogo de artículos comercializados por la empresa: `id_producto` (PK), nombre, descripción, stock y precio. El campo stock se actualiza constantemente a partir de los movimientos registrados.
- **MOVIMIENTO (STOCK):** entidad central que registra cada entrada o salida de inventario: `id_movimiento` (PK), fecha, cantidad, tipo, `id_usuario` (FK) e `id_producto` (FK). Cada movimiento queda vinculado tanto al usuario que lo generó como al producto afectado.
- **MOVIMIENTO (COMPRA-VENTA):** registra las transacciones comerciales propiamente dichas: `id_mov_cv` (PK), fecha, tipo (Compra/Venta), total_importe e `id_distribuidor` (FK), conectándose además con los movimientos de stock mediante la denominada conexión de movimientos.
- **DISTRIBUIDOR:** almacena la información de los proveedores de materia prima: `id_distribuidor` (PK), nombre, contacto y dirección.

8.2 Relaciones y cardinalidades

Relación	Cardinalidad	Significado
USUARIO ↔ MOVIMIENTO (STOCK)	1 a N	Un usuario puede generar muchos movimientos de stock
PRODUCTO ↔ MOVIMIENTO (STOCK)	1 a N	Un producto puede estar involucrado en muchos movimientos
MOVIMIENTO (STOCK) ↔ MOVIMIENTO (COMPRA-VENTA)	N a N (conexión de movimientos)	Varios movimientos de stock pueden asociarse a una o varias transacciones de compra-venta
DISTRIBUIDOR ↔ MOVIMIENTO (COMPRA-VENTA)	1 a N	Un distribuidor puede estar asociado a muchas transacciones de compra

Sobre las claves primarias y foráneas

En el diagrama, PK identifica la clave primaria de cada entidad (el identificador único de cada registro), mientras que FK identifica una clave foránea, es decir, una referencia a la clave primaria de otra entidad. Estas relaciones son las que, en Oracle Database, se implementan mediante restricciones de integridad referencial (FOREIGN KEY constraints), garantizando que, por ejemplo, no pueda registrarse un movimiento de stock que referencie a un producto inexistente.

9. Diagrama de Secuencia

A diferencia del diagrama de iteración presentado en la sección 7 —centrado en el proceso de venta—, este diagrama de secuencia describe dos procesos distintos y consecutivos: el inicio de sesión del usuario (Process 1) y el registro de una entrada de inventario (Process 2), detallando las interacciones de bajo nivel entre el cliente y el backend.

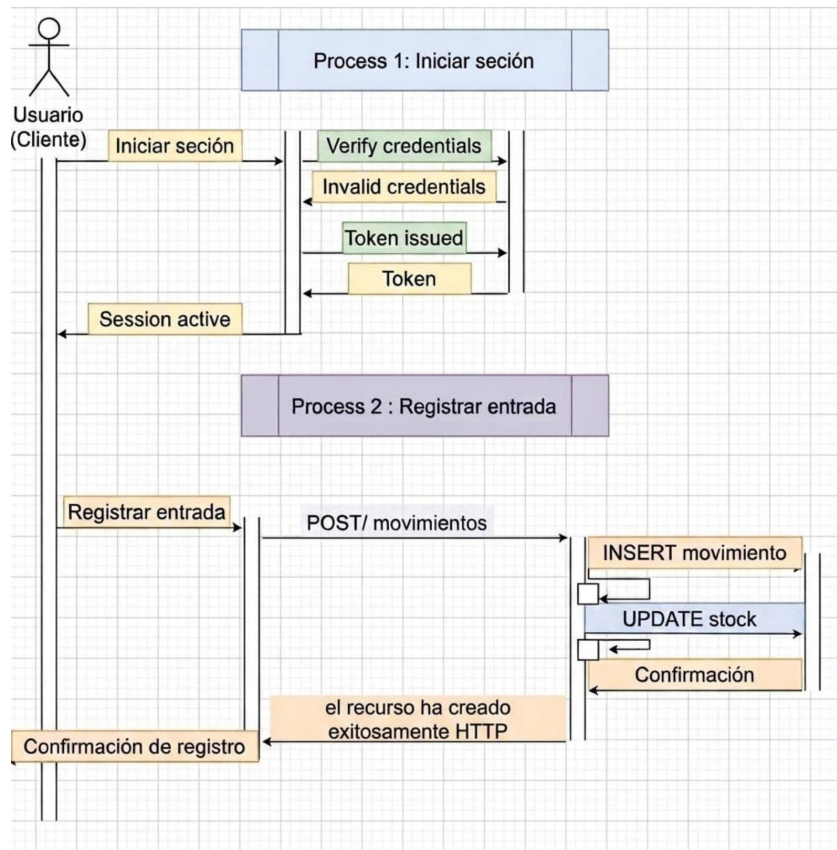


Diagrama de secuencia de los procesos de inicio de sesión (Process 1) y registro de entrada de inventario (Process 2) del sistema Crispy.

9.1 Process 1 — Iniciar sesión

El proceso de autenticación comienza cuando el Usuario (Cliente) envía sus credenciales mediante la acción Iniciar sesión. El sistema verifica dichas credenciales (Verify credentials); si son incorrectas, se retorna un mensaje de Invalid credentials, lo que permite al usuario intentar nuevamente. Si las credenciales son válidas, el sistema emite un token de sesión (Token issued), que se entrega al cliente como Token, habilitando un estado de Session active que se mantiene mientras el token siga siendo válido.

- **Iniciar sesión:** el usuario envía sus credenciales (usuario y contraseña) al sistema.
- **Verify credentials / Invalid credentials:** el sistema valida las credenciales contra la tabla USUARIO; si no coinciden, responde con un error de credenciales inválidas.
- **Token issued / Token:** una vez validadas las credenciales, el sistema genera y entrega un token de sesión al cliente.
- **Session active:** el cliente queda con una sesión activa, lista para realizar operaciones autenticadas.

9.2 Process 2 — Registrar entrada

Una vez autenticado, el usuario puede ejecutar el segundo proceso: registrar una entrada de inventario. La solicitud se envía mediante un método HTTP POST hacia el endpoint /movimientos. El backend ejecuta primero una instrucción INSERT movimiento para registrar el nuevo movimiento, seguida de una instrucción UPDATE stock para reflejar el incremento del inventario disponible. Tras recibir la Confirmación desde la base de datos, el backend responde al cliente indicando que el recurso ha creado exitosamente HTTP (es decir, con un código de estado HTTP 201 Created), y finalmente el cliente recibe la Confirmación de registro.

- **Registrar entrada / POST /movimientos:** el cliente, ya autenticado, envía los datos de la nueva entrada de inventario al backend.
- **INSERT movimiento / UPDATE stock:** el backend ejecuta de forma secuencial el registro del movimiento y la actualización del stock correspondiente en Oracle Database.
- **Confirmación:** la base de datos confirma al backend que ambas operaciones se completaron correctamente.
- **el recurso ha creado exitosamente HTTP / Confirmación de registro:** el backend responde con el código HTTP correspondiente a una creación exitosa, y el cliente recibe la confirmación final.

Relación con el diagrama de iteración

Este diagrama de secuencia complementa al diagrama de iteración de la sección 7: mientras aquel se enfoca en el flujo de una venta desde la perspectiva del Comprador/Vendedor, este se enfoca en los procesos de autenticación y de registro de entradas de inventario, generalmente asociados al rol de Administrador o Distribuidor.

10. Diccionario de Datos

El diccionario de datos documenta, de forma detallada, cada uno de los atributos que componen las entidades identificadas en el diagrama de entidad-relación (sección 8), especificando su tipo de dato, su función dentro del modelo y las restricciones que aplican sobre cada campo. Este documento es una referencia indispensable tanto para la creación de las tablas en Oracle Database como para el desarrollo del backend que interactúa con ellas.

10.1 Tabla USUARIO

Campo	Tipo de dato (Oracle)	Descripción
id_usuario	NUMBER (PK)	Identificador único del usuario, autogenerado mediante secuencia o IDENTITY
nombre	VARCHAR2(100)	Nombre completo de la persona
username	VARCHAR2(50)	Nombre de usuario utilizado para iniciar sesión
password	VARCHAR2(255)	Contraseña almacenada de forma cifrada (hash)
rol	VARCHAR2(20)	Rol del usuario: Administrador, Distribuidor o Comprador/Vendedor

10.2 Tabla PRODUCTO

Campo	Tipo de dato (Oracle)	Descripción
id_producto	NUMBER (PK)	Identificador único del producto
nombre	VARCHAR2(150)	Nombre comercial del producto
descripcion	VARCHAR2(500)	Descripción detallada del producto
stock	NUMBER	Cantidad disponible actualmente en inventario
precio	NUMBER(10,2)	Precio unitario de venta del producto

10.3 Tabla MOVIMIENTO (STOCK)

Campo	Tipo de dato (Oracle)	Descripción
id_movimiento	NUMBER (PK)	Identificador único del movimiento de stock
fecha	DATE / TIMESTAMP	Fecha y hora en que se registró el movimiento
cantidad	NUMBER	Cantidad de producto afectada por el movimiento
tipo	VARCHAR2(20)	Tipo de movimiento: entrada o salida de inventario

Campo	Tipo de dato (Oracle)	Descripción
id_usuario	NUMBER (FK)	Referencia al usuario que generó el movimiento
id_producto	NUMBER (FK)	Referencia al producto afectado por el movimiento

10.4 Tabla MOVIMIENTO (COMPRA-VENTA)

Campo	Tipo de dato (Oracle)	Descripción
id_mov_cv	NUMBER (PK)	Identificador único de la transacción de compra-venta
fecha	DATE / TIMESTAMP	Fecha y hora de la transacción
tipo	VARCHAR2(20)	Tipo de transacción: Compra o Venta
total_importe	NUMBER(10,2)	Monto total de la transacción
id_distribuidor	NUMBER (FK)	Referencia al distribuidor asociado a la transacción, cuando aplica

10.5 Tabla DISTRIBUIDOR

Campo	Tipo de dato (Oracle)	Descripción
id_distribuidor	NUMBER (PK)	Identificador único del distribuidor
nombre	VARCHAR2(150)	Nombre o razón social del distribuidor
contacto	VARCHAR2(100)	Persona o medio de contacto del distribuidor
direccion	VARCHAR2(255)	Dirección física del distribuidor

Convenciones utilizadas

PK identifica la clave primaria de cada tabla; FK identifica una clave foránea que referencia la clave primaria de otra tabla. Los tipos de datos mostrados corresponden a la nomenclatura propia de Oracle Database (por ejemplo, VARCHAR2 en lugar de VARCHAR, y NUMBER como tipo numérico genérico para enteros y decimales).

11. Código del Sistema

El backend del sistema Crispy se implementa en Node.js utilizando únicamente el módulo nativo http, sin frameworks adicionales, con el propósito de ilustrar de forma didáctica cómo se construye un servidor web desde sus fundamentos. En esta primera versión, los datos de productos se simulan mediante un arreglo en memoria, en lugar de conectarse a una instancia real de Oracle Database, lo que permite centrar la explicación en la lógica de enrutamiento y procesamiento HTTP. Más adelante, en la sección 11.5, se complementa este código con una versión que sí establece conexión real con Oracle mediante el driver oficial oracledb.

11.1 Bloque 1 y 2 — Infraestructura y configuración de puertos



Bloque 1: importación de módulos (infraestructura). Bloque 2: configuración de los puertos de red.

El primer bloque importa el módulo nativo http de Node.js, encargado de crear el servidor web. La línea comentada que referencia oracledb indica el punto donde se incorporaría el controlador nativo de Oracle en una implementación con conexión real a base de datos. El segundo bloque define dos constantes de puerto: PORT (8080), utilizado por el servidor de aplicaciones Node.js, y DB_PORT (1521), el puerto estándar del listener de Oracle Database.

```
const http = require('http');  
// const oracledb = require('oracledb'); // Controlador nativo de Oracle  
  
const PORT = 8080;  
const DB_PORT = 1521;
```

11.2 Bloque 3 y 4 — Modelo de datos simulado y servidor HTTP



Bloque 3: simulación del modelo de datos (esquema de tablas). Bloque 4: creación del servidor HTTP y enrutamiento básica.

El bloque 3 simula, mediante un arreglo de objetos JavaScript, lo que en una base de datos real sería el contenido de la tabla PRODUCTO: cada objeto incluye un identificador, un nombre comercial, un SKU, un precio y el stock disponible. El bloque 4 inicia la creación del servidor HTTP mediante `http.createServer()`, recibiendo una función de callback que se ejecuta cada vez que llega una petición (`req`) y permite construir una respuesta (`res`).

```
let productos = [  
  { id: 1, nombre: "Derivados de Amaranto Crispy", sku: "AMAR-CRISP-01", precio:  
    25.00, stock: 150 },  
  { id: 2, nombre: "Cereal de Amaranto con Miel", sku: "AMAR-MIEL-02", precio: 35.00,  
    stock: 90 },  
];  
  
const server = http.createServer((req, res) => {  
  // Aquí adentro se procesa lo que pide el navegador  
});
```

11.3 Bloque 5 y 6 — Encabezados HTTP y lógica transaccional

CODIGO

BLOQUE 5: CONFIGURACIÓN DE ENCABEZADOS (HEADERS) Y TIPO DE CONTENIDO

```
res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
```

BLOQUE 6: LÓGICA TRANSACCIONAL (SIMULACIÓN DE UPDATE E INSERT DE ORACLE)

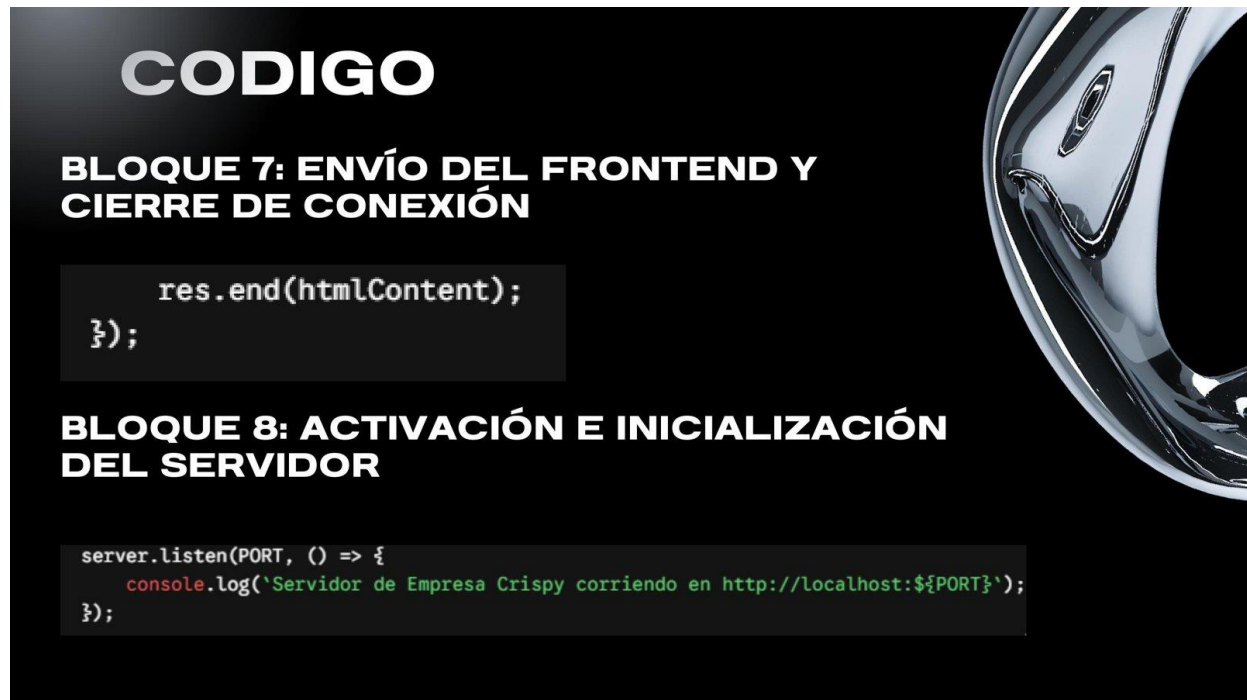
```
// Ejemplo de flujo cuando el distribuidor altera stock:  
// productos[0].stock += cantidad;  
  
// Ejemplo de flujo cuando el comprador vende:  
// productos[0].stock -= cantidad;
```

Bloque 5: configuración de encabezados (headers) y tipo de contenido. Bloque 6: lógica transaccional, simulación de UPDATE e INSERT de Oracle.

El bloque 5 configura el encabezado de respuesta HTTP, indicando un código de estado 200 (éxito) y especificando que el contenido devuelto será HTML codificado en UTF-8. El bloque 6 ilustra, de forma comentada, cómo se modificaría el stock de un producto según el tipo de operación: cuando un distribuidor registra una entrada, la cantidad se suma al stock (+); cuando un comprador realiza una venta, la cantidad se resta del stock (-). Estas operaciones, en una implementación real conectada a Oracle, se traducirían en instrucciones SQL UPDATE sobre la tabla PRODUCTO.

```
res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });  
  
// Ejemplo de flujo cuando el distribuidor altera stock:  
// productos[0].stock += cantidad;  
  
// Ejemplo de flujo cuando el comprador vende:  
// productos[0].stock -= cantidad;
```

11.4 Bloque 7 y 8 — Envío de respuesta e inicialización del servidor



Bloque 7: envío del frontend y cierre de conexión. Bloque 8: activación e inicialización del servidor.

El bloque 7 cierra el ciclo de la petición HTTP: `res.end(htmlContent)` envía al navegador el contenido HTML generado dinámicamente y finaliza la conexión. El bloque 8 pone en marcha el servidor mediante `server.listen(PORT, ...)`, indicando que debe escuchar conexiones entrantes en el puerto definido previamente (8080), e imprime en consola un mensaje de confirmación una vez que el servidor está activo.

```
res.end(htmlContent);  
});  
  
server.listen(PORT, () => {  
  console.log(`Servidor de Empresa Crispy corriendo en http://localhost:${PORT}`);  
});
```

11.5 Complemento: conexión real a Oracle Database con el driver oracledb

El código presentado en las secciones 11.1 a 11.4 simula el comportamiento del sistema utilizando un arreglo en memoria, lo cual es perfectamente válido con fines didácticos para entender la lógica de enrutamiento HTTP sin depender de una base de datos activa. Sin embargo, en una implementación real, esas mismas operaciones deben traducirse en instrucciones SQL ejecutadas contra una instancia de Oracle Database mediante el driver oficial de Node.js, `oracledb`, mantenido por Oracle Corporation. A continuación se presenta cómo se adaptaría cada bloque de código para trabajar con una conexión real.

11.5.1 Instalación del driver

El primer paso es instalar el paquete `oracledb` desde `npm`. Este driver puede operar en modo Thin (puramente en JavaScript, sin dependencias adicionales) o en modo Thick (requiere las Oracle Instant Client libraries), siendo el modo Thin el predeterminado y el más sencillo de configurar para la mayoría de los casos.

```
npm install oracledb
```

11.5.2 Configuración de la conexión (sustituye al Bloque 1 y 2)

En lugar de simular la infraestructura, se importa el driver `oracledb` y se definen los parámetros de conexión: usuario, contraseña y `connectString` (cadena que combina el host, el puerto y el nombre del servicio o pluggable database, por ejemplo XEPDB1 en una instalación de Oracle Database XE).

```
const http = require('http');
const oracledb = require('oracledb');

const PORT = 8080;

const dbConfig = {
  user: "crispy_app",
  password: process.env.DB_PASSWORD, // nunca escribir la contraseña directamente en
  // el código
  connectString: "localhost:1521/XEPDB1",
};
```

11.5.3 Creación de un pool de conexiones (sustituye al modelo simulado)

En vez de simular productos mediante un arreglo en memoria, una aplicación real debe crear un pool de conexiones al iniciar el servidor. Un pool reutiliza conexiones ya establecidas en lugar de abrir y cerrar una conexión nueva en cada petición, lo cual mejora considerablemente el rendimiento bajo carga.

```
async function iniciarPool() {
  await oracledb.createPool({
    user: dbConfig.user,
    password: dbConfig.password,
    connectString: dbConfig.connectString,
    poolMin: 2,
    poolMax: 10,
    poolIncrement: 1,
  });
  console.log("Pool de conexiones a Oracle Database creado correctamente.");
}
```

11.5.4 Consulta del inventario (sustituye a la lectura del arreglo productos)

Donde antes se leía directamente el arreglo productos, ahora se obtiene una conexión del pool y se ejecuta una instrucción SELECT contra la tabla PRODUCTO, devolviendo los resultados como un arreglo de filas.

```
async function obtenerProductos() {
  let connection;
  try {
    connection = await oracledb.getConnection();
    const resultado = await connection.execute(
      `SELECT id_producto, nombre, stock, precio FROM producto ORDER BY id_producto`,
      [],
      { outFormat: oracledb.OUT_FORMAT_OBJECT }
    );
    return resultado.rows;
  } finally {
    if (connection) {
      await connection.close(); // libera la conexión de vuelta al pool
    }
  }
}
```

11.5.5 Registro transaccional de una venta (sustituye al Bloque 6)

Esta es la adaptación más importante: el registro de un movimiento (INSERT) y la actualización del stock (UPDATE) deben ejecutarse dentro de la misma transacción, confirmando ambos cambios con connection.commit() únicamente si los dos pasos se completan sin errores, o revirtiéndolos con connection.rollback() si alguno falla. Esto evita que quede un movimiento registrado sin su correspondiente ajuste de stock.

```
async function registrarVenta(idProducto, idUsuario, cantidad) {
  let connection;
  try {
    connection = await oracledb.getConnection();

    // 1. Verificar que existe stock suficiente
    const stockActual = await connection.execute(
      `SELECT stock FROM producto WHERE id_producto = :id`,
      { id: idProducto }
    );
    const disponible = stockActual.rows[0][0];
    if (disponible < cantidad) {
      throw new Error("Stock insuficiente para completar la venta.");
    }
  }
}
```

```
}

// 2. INSERT del movimiento (equivalente al Bloque 6 simulado)
await connection.execute(
  `INSERT INTO movimiento (id_movimiento, fecha, cantidad, tipo, id_usuario,
id_producto)
  VALUES (seq_movimiento.NEXTVAL, SYSDATE, :cantidad, 'salida', :idUserio,
:idProducto)`,
  { cantidad, idUsuario, idProducto }
);

// 3. UPDATE del stock (descuento por venta)
await connection.execute(
  `UPDATE producto SET stock = stock - :cantidad WHERE id_producto =
:idProducto`,
  { cantidad, idProducto }
);

// 4. Confirmar ambos cambios como una sola unidad atómica
await connection.commit();
return { ok: true, mensaje: "Venta registrada correctamente." };

} catch (error) {
  if (connection) {
    await connection.rollback(); // revierte INSERT y UPDATE si algo falló
  }
  throw error;
} finally {
  if (connection) {
    await connection.close();
  }
}
}
```

11.5.6 Cierre ordenado del servidor y del pool

Finalmente, al inicializar el servidor HTTP, conviene crear primero el pool de conexiones y, al recibir una señal de cierre del proceso, cerrar también el pool de forma ordenada, liberando todas las conexiones abiertas hacia Oracle Database.

```
const server = http.createServer((req, res) => {
  // Enrutamiento HTTP: aquí se invocarían obtenerProductos() o registrarVenta()
});

iniciarPool().then(() => {
  server.listen(PORT, () => {
    console.log(`Servidor de Empresa Crispy corriendo en http://localhost:${PORT}`);
```

```
});  
});  
  
process.on('SIGTERM', async () => {  
  await oracledb.getPool().close(10); // espera hasta 10s a que terminen conexiones  
  activas  
  process.exit(0);  
});
```

Buena práctica de seguridad

Nótese que la contraseña de conexión se obtiene de `process.env.DB_PASSWORD` en lugar de escribirse directamente en el código fuente. Mantener las credenciales en variables de entorno, o en un gestor de secretos, evita que queden expuestas si el código se comparte o se sube a un repositorio.

12. Roles del Sistema

Antes de presentar las capturas de pantalla de la aplicación en funcionamiento, conviene recapitular los tres roles de usuario que el sistema reconoce, ya que cada uno de ellos tiene acceso a una vista distinta y a un conjunto de permisos diferenciado sobre la base de datos.



Recordatorio de los tres roles del sistema Crispy: Administrador, Distribuidor y Comprador/Vendedor.

12.1 Administrador

El Administrador cuenta con visión global sobre el negocio: puede consultar métricas financieras agregadas, como los ingresos por ventas, los egresos por compras de materia prima y la ganancia neta resultante, así como revisar el inventario completo de productos.

12.2 Distribuidor

El Distribuidor tiene permisos restringidos exclusivamente a la inserción y actualización de existencias: puede registrar entradas de materia prima o producto terminado, lo que incrementa el stock disponible, pero no tiene visibilidad sobre las utilidades del negocio ni puede modificar los precios de los productos.

12.3 Comprador/Vendedor

El Comprador/Vendedor interactúa con la interfaz de punto de venta (POS), registrando la salida de productos terminados como resultado de una venta, lo que reduce el stock disponible y genera el correspondiente ingreso financiero para el negocio.

Rol	Acción principal sobre el inventario	Visibilidad de métricas financieras
Administrador	Consulta global; no modifica directamente el stock	Completa (ingresos, egresos, utilidades)
Distribuidor	Incrementa el stock (entradas de materia prima)	Ninguna
Comprador/Vendedor	Reduce el stock (salidas por venta)	Solo el registro de su propia transacción

Principio de seguridad aplicado

Esta segmentación de permisos por rol es una aplicación directa del principio de privilegio mínimo: cada actor del sistema solo puede ejecutar las operaciones estrictamente necesarias para cumplir su función, reduciendo el riesgo de modificaciones accidentales o no autorizadas sobre datos sensibles como las utilidades del negocio.

13. Capturas de Pantalla del Sistema

Para ilustrar el funcionamiento real de la aplicación, a continuación se presentan capturas de pantalla del sistema Crispy operando bajo cada uno de los tres roles definidos: Administrador, Distribuidor y Comprador/Vendedor. Estas capturas permiten verificar de forma concreta cómo se traducen, en la interfaz de usuario, las reglas de negocio y los modelos de datos descritos en las secciones anteriores.

13.1 Panel del Administrador

El panel del Administrador despliega tres indicadores financieros clave —Ingresos de Venta, Egresos de Compra y Ganancia Neta (Utilidades)— calculados a partir de los movimientos de compra-venta registrados en el sistema. Debajo de estos indicadores se presenta el inventario general, que simula directamente el contenido de la tabla PRODUCTO en Oracle Database, mostrando el identificador, nombre comercial, SKU, precio unitario y stock disponible de cada artículo.



Panel General del Administrador: indicadores de ingresos, egresos y ganancia neta, junto con el inventario general de productos.

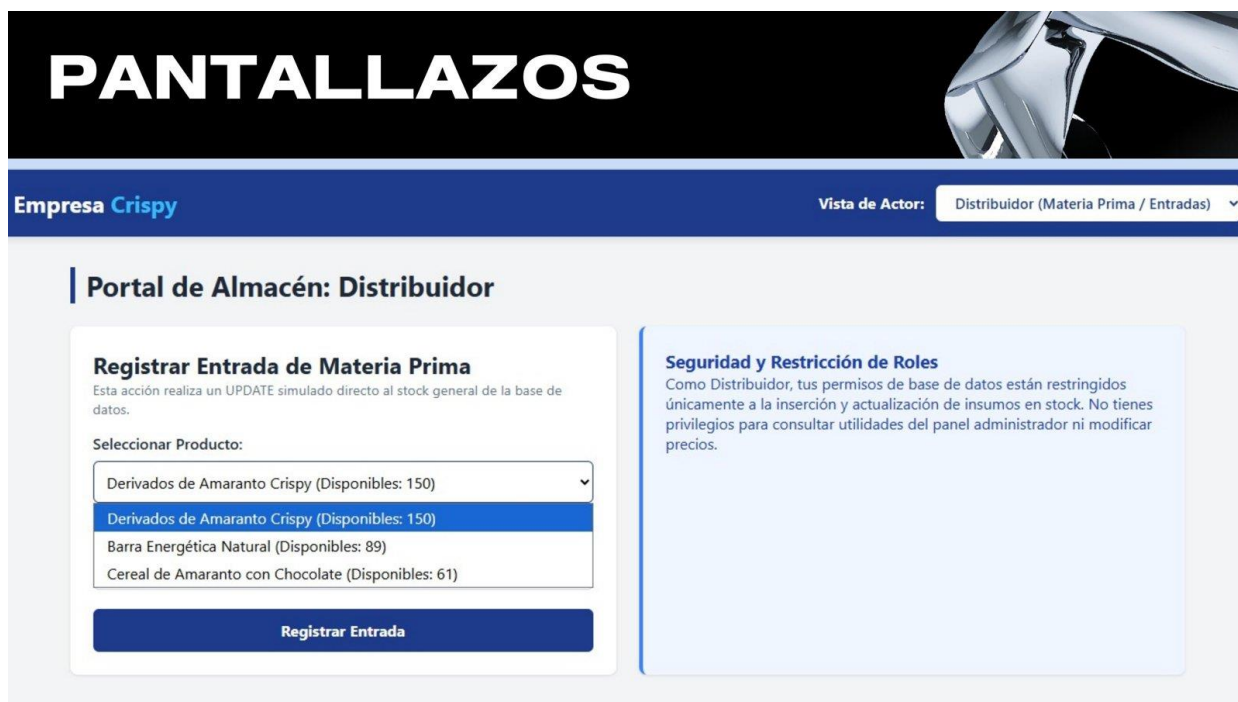


Continuación del panel del Administrador, mostrando el detalle completo del inventario general con los tres productos registrados.

Resulta interesante observar que, en la captura presentada, la Ganancia Neta aparece en negativo (\$-7.50), lo cual evidencia que, en el momento de la captura, los egresos por compra de materia prima superaron a los ingresos por venta. Este tipo de indicador es valioso precisamente porque permite a la administración detectar con rapidez situaciones de margen negativo y tomar decisiones correctivas, como ajustar precios o renegociar costos con los distribuidores.

13.2 Panel del Distribuidor

El portal del Distribuidor está diseñado conforme al principio de privilegio mínimo descrito en la sección 12: únicamente permite seleccionar un producto del catálogo y registrar una entrada de materia prima, indicando la cantidad entrante. El sistema le recuerda explícitamente, mediante un panel informativo, que sus permisos están restringidos a la inserción y actualización de stock, sin acceso a las utilidades del panel administrativo ni a la modificación de precios.



PANTALLAZOS

Empresa Crispy

Vista de Actor: Distribuidor (Materia Prima / Entradas)

Portal de Almacén: Distribuidor

Registrar Entrada de Materia Prima

Esta acción realiza un UPDATE simulado directo al stock general de la base de datos.

Seleccionar Producto:

Derivados de Amaranto Crispy (Disponibles: 150)

Derivados de Amaranto Crispy (Disponibles: 150)

Barra Energética Natural (Disponibles: 89)

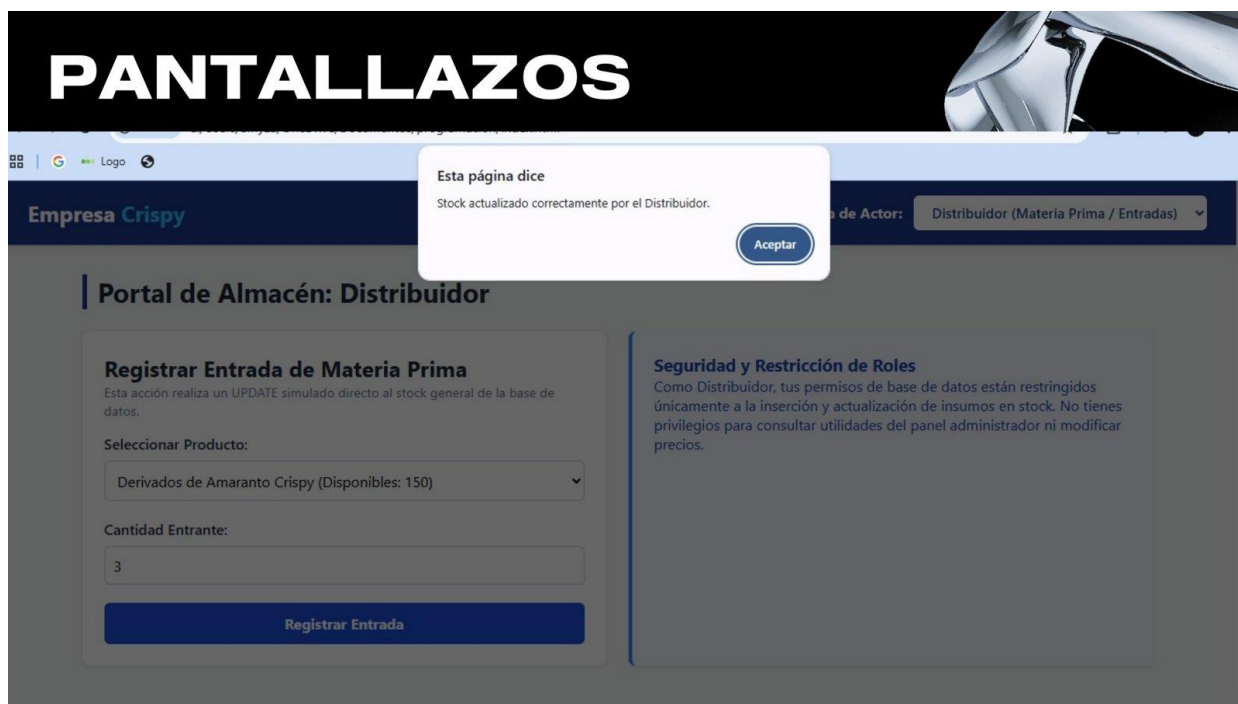
Cereal de Amaranto con Chocolate (Disponibles: 61)

Registrar Entrada

Seguridad y Restricción de Roles

Como Distribuidor, tus permisos de base de datos están restringidos únicamente a la inserción y actualización de insumos en stock. No tienes privilegios para consultar utilidades del panel administrador ni modificar precios.

Portal de Almacén del Distribuidor: formulario para registrar entrada de materia prima y panel de seguridad y restricción de roles.



PANTALLAZOS

Empresa Crispy

Vista de Actor: Distribuidor (Materia Prima / Entradas)

Portal de Almacén: Distribuidor

Registrar Entrada de Materia Prima

Esta acción realiza un UPDATE simulado directo al stock general de la base de datos.

Seleccionar Producto:

Derivados de Amaranto Crispy (Disponibles: 150)

Derivados de Amaranto Crispy (Disponibles: 150)

Barra Energética Natural (Disponibles: 89)

Cereal de Amaranto con Chocolate (Disponibles: 61)

Cantidad Entrante:

3

Registrar Entrada

Seguridad y Restricción de Roles

Como Distribuidor, tus permisos de base de datos están restringidos únicamente a la inserción y actualización de insumos en stock. No tienes privilegios para consultar utilidades del panel administrador ni modificar precios.

Confirmación del sistema tras registrar una entrada de 3 unidades de "Derivados de Amaranto Crispy" por parte del Distribuidor.

La captura de confirmación evidencia, en términos funcionales, la ejecución exitosa de la instrucción UPDATE sobre el stock del producto seleccionado: el mensaje "Stock actualizado correctamente por el

Distribuidor” es la traducción, a nivel de interfaz, de la confirmación SQL que se describió en el diagrama de iteración de la sección 7.

13.3 Panel del Comprador/Vendedor (POS)

Finalmente, la interfaz de Punto de Venta (POS) asociada al rol de Comprador/Vendedor permite seleccionar un artículo comercial y registrar la cantidad a comprar, completando la venta mediante el botón correspondiente. A la derecha de la pantalla, un panel de Auditoría de Movimientos de la Sesión muestra en tiempo real el registro cronológico de las operaciones realizadas, incluyendo tanto entradas registradas por el Distribuidor como ventas completadas por el Comprador, cada una con su marca de tiempo y el efecto financiero correspondiente (ingreso o costo de inversión).



Punto de Venta (POS) del Comprador/Vendedor, con el formulario de despacho de producto terminado y la auditoría de movimientos de la sesión.

Este panel de auditoría resulta especialmente valioso desde el punto de vista de la trazabilidad: permite reconstruir, sin necesidad de consultar directamente la base de datos, la secuencia de eventos que explica el estado actual del inventario y de las métricas financieras mostradas en el panel del Administrador.

14. Conclusiones

El sistema de control de inventario desarrollado para la Empresa Crispy ilustra, de manera integral, cómo Oracle Database puede emplearse como motor central de un sistema de información de pequeña y mediana escala. A lo largo de este documento se repasó la historia de Oracle Corporation, desde su fundación en 1977 hasta su consolidación como referente mundial en la gestión de bases de datos

relacionales, y se describió el proceso de instalación de Oracle Database Express Edition como punto de partida práctico para cualquier proyecto académico o de desarrollo a pequeña escala.

Desde el punto de vista de diseño, los distintos diagramas presentados —arquitectura general, despliegue, iteración y secuencia— mostraron cómo un mismo sistema puede analizarse desde perspectivas complementarias: los componentes lógicos involucrados, su distribución física sobre dispositivos y servidores reales, y el detalle paso a paso de los procesos críticos de venta, autenticación y registro de inventario. El diagrama de entidad-relación y el diccionario de datos asociado tradujeron estos conceptos en un modelo de base de datos concreto, compuesto por las entidades USUARIO, PRODUCTO, MOVIMIENTO (STOCK), MOVIMIENTO (COMPRA-VENTA) y DISTRIBUIDOR.

En el plano de la implementación, se presentó el código del servidor en Node.js en su versión simulada —útil con fines didácticos para comprender la lógica de enrutamiento HTTP sin depender de una base de datos activa—, así como su complemento con conexión real a Oracle Database mediante el driver oficial oracledb, incluyendo el uso de un pool de conexiones y el manejo transaccional con commit y rollback para garantizar que el registro de movimientos y la actualización de stock ocurran siempre como una unidad atómica.

Finalmente, las capturas de pantalla del sistema en funcionamiento permitieron verificar que la segmentación de roles —Administrador, Distribuidor y Comprador/Vendedor— efectivamente se traduce en vistas e interfaces diferenciadas, aplicando de forma consistente el principio de privilegio mínimo a lo largo de toda la aplicación. En conjunto, este proyecto demuestra que, incluso en una implementación de alcance académico, es posible construir un sistema completo y coherente que combine buenas prácticas de modelado de datos, diseño de arquitectura por capas y manejo transaccional confiable sobre Oracle Database.